pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

An Orchestrator and Specialists

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: an orchestrator and specialists

Week 3: many agents, kept honest

Day 11	an orchestrator and specialists
Day 12	memory and state
Day 13	evals
Day 14	guardrails and cost control

Today

1. Why split: what every request carries
2. specialist and route_table
3. Handoffs against agents as tools
4. The allow-list, asserted
5. route_table, and the desk brief
6. Parallel tool requests, once
7. What the orchestrator costs
8. The patterns drawn, Excel, Git, AI check

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

On the synthetic book: numbers from tools, described

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

On every agent's output: text, a saved run, a brief

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

Why split: what every request carries

```
# characters of tool schemas each agent sends with every request
macro                    2,974 characters of tool schemas
credit analyst           1,481 characters of tool schemas
risk                     6,749 characters of tool schemas
PM                           2 characters of tool schemas
one agent, every tool   11,204 characters of tool schemas
```

Shorter requests

only its own tools

Fewer wrong tools

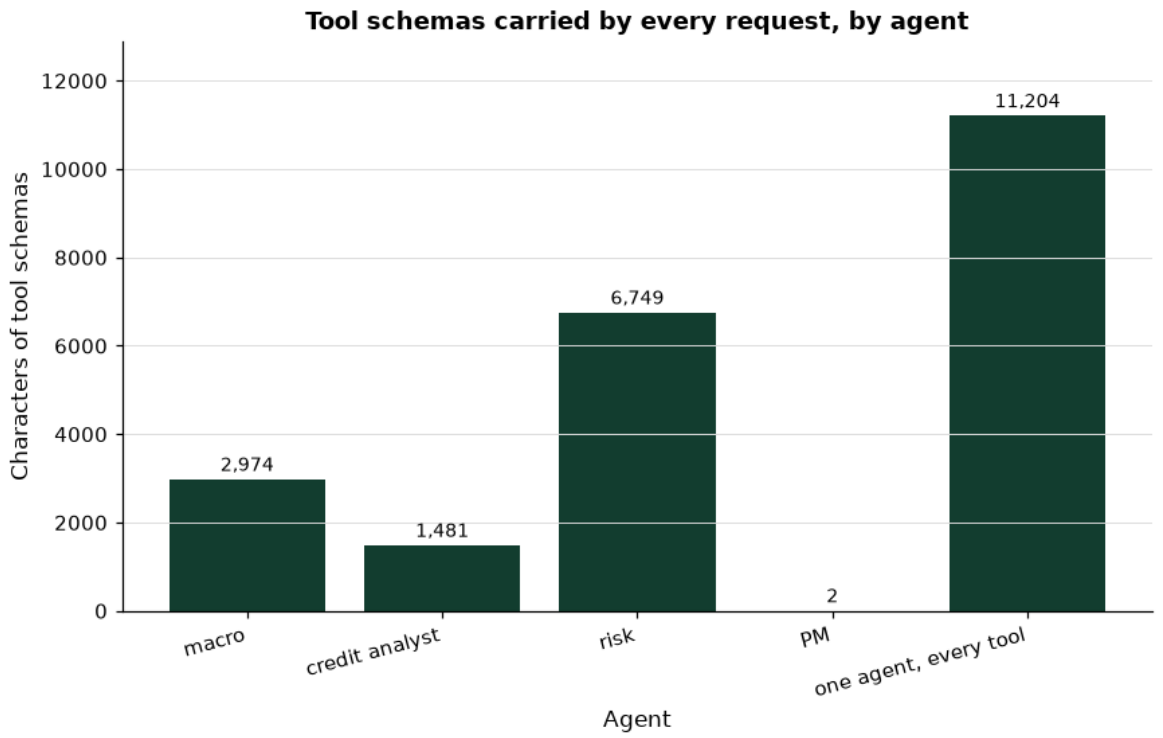
less to choose among

One job each

instructions for one job

20 tools in one agent; the covenant tool maps 11 sections of Article 4 for the credit analyst.

What each agent carries



One agent with every tool sends 11,204 characters of tool schemas on every request; the PM's empty list is 2.

How a specialist works

The problem	An agent should hold only the tools its job needs
The method	Build with sdk_agent, then raise a ValueError if the tool names are not the list's keys
The check	A typo or a stray tool stops at build time, not in the middle of a run
You control	The allow-list and the instructions
What comes out	An agent whose tools are the list, and route_table: who did each step

Four specialists, each from an allow-list

```
risk = agentkit.specialist("risk", RISK, RISK_INSTRUCTIONS, aclient, model, 11).clone(
    output_type=agentkit.DeskAnswer, handoff_description=DESCRIPTIONS["risk"])
```

```
macro           tools  6; answer DeskAnswer
credit analyst  tools  3; answer CreditAnswer
risk            tools 11; answer DeskAnswer
PM              tools  0; answer text
```

route_table(result): one row per step: the agent, and a tool request, result, handoff, or answer.

specialist raises when the tools differ from the list. 54 passed with the day's two tests.

How the two ways work

The link	Both ways join one agent to another through a tool
A handoff	transfer_to_<name>: the specialist reads the conversation, and its answer ends the run
An agent as tool	The orchestrator writes an input; the specialist runs fresh; its answer is the result
You control	Which specialists are offered, their descriptions, each step limit
What comes out	Every request in the exchanges; the route table shows the active agent only

Handoffs against agents as tools

A handoff passes the conversation

transfer_to_<name>: one tool per specialist

the specialist reads the whole conversation

its answer ends the run

the orchestrator does not speak again

An agent as a tool keeps control

one tool per specialist, an input it writes

the specialist starts a run of its own

its answer comes back as a tool result

the orchestrator asks several, then answers

```
position DV01 of the book: 306,996.49 dollars per bp; the 10 Yr on 2026-10-02: 5.28 percent
```

The question needs two specialists: the book's DV01 (risk) and the 10 Yr in the file (macro).

Every check passes; the yield is wrong

4.0

percent: yield_to_maturity at par

5.28

percent: the file's 10 Yr

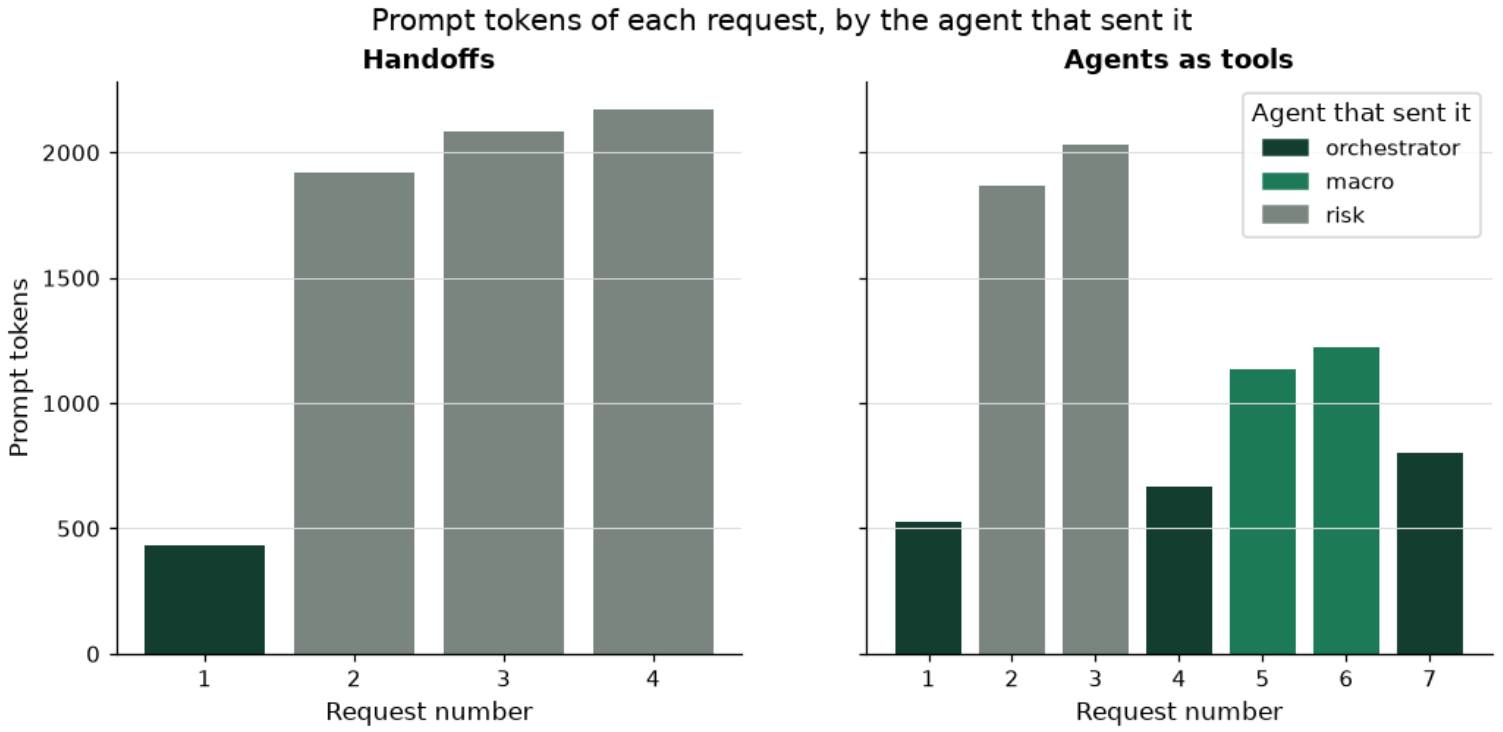
```
Total position DV01 in dollars of the synthetic holdings file: 306,996.493904 usd; pandas 306,996.493904; the...
10 Yr Treasury par yield on 2026-10-02: 4.0 pct; pandas 5.280000; the same at 2 decimals: no
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 4 requests, 6,610 prompt and 249 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

A 4 percent bond priced at 100 yields 4 percent, whatever the curve says. The tool was asked the wrong question.

One specialist received the whole question, and it needed two. The keys caught it; the checks could not.

Prompt tokens of each request



The risk specialist's requests are the largest in both runs: it carries eleven tools. The orchestrator carries none.

How the allow-list check works

The claim	An allow-list says what an agent can do
Check one	specialist, once, when the agent is built
Check two	The assertion cell, on the agent as it stands
Check three	The tools offered in the request bodies it sent: what the model saw
What comes out	A failure at build time, never a surprise in a run

specialist and the allow-list, asserted

```
ValueError: risk's tools are ['dv01'], not the allow-list ['dv01_per_100']
```

```
macro           holds exactly its allow-list: 6 tools
credit analyst  holds exactly its allow-list: 3 tools
risk            holds exactly its allow-list: 11 tools
PM              holds exactly its allow-list: 0 tools
risk            offered in its saved requests: 11 tools, the allow-list exactly
macro           offered in its saved requests: 6 tools, the allow-list exactly
```

The orchestrator offered only handoffs and agent tools: it holds no data tool, so it fetches no number itself.

How route_table works

The reader	route_table reads result.new_items: one row for each step
The row	Step, agent, kind (request, result, handoff, answer), content
Its limit	It sees the outer run: an agent as a tool is one request and one result
The other view	request_table reads the exchanges and names the agent from the system message
What comes out	Two tables of one run: by step and by request

route_table: who handled which step

Handoffs: the specialist's steps are the run's

```
1 orchestrator handoff transfer_to_risk
2 orchestrator handoff result to risk
3 risk tool request portfolio_totals
4 risk tool result {"as_of": "2026-09-30", "bonds": 200, "dts_years_bp": 1064.114013, "duration_...
5 risk tool request yield_to_maturity
6 risk tool result {"tool_call_id": "call_ifpnXWdtxKgXkTYDvd1JEXut", "yield_pct": 4.0}
7 risk answer {"text":"As of 2026-09-30, the total position DV01 in dollars of the syntheti...
```

Agents as tools: the specialists run inside a tool

```
1 orchestrator tool request risk
2 orchestrator tool result {"figures": [{"name": "Total position DV01 in dollars", "tool_call_id"...
3 orchestrator tool request macro
4 orchestrator tool result {"figures": [{"name": "10 Yr Treasury par yield on 2026-10-02", "tool_...
5 orchestrator answer As of 2026-09-30, the total position DV01 in dollars of the synthetic holdi...
```

The route table says who handled each step; the request table, read from the exchanges, says who sent each request.

How the desk brief works

The pattern	Two patterns joined: three workers as tools, then a handoff to the PM
The workers	The orchestrator collects their answers and keeps control
The writer	The PM has no tool; it writes from the conversation in front of it
You control	The three questions, the tools, the handoff, the PM's instructions
What comes out	A brief whose every number is in a specialist's answer

The desk brief: three specialists, then the PM

4	2	4	2	1
orchestrator	macro	credit analyst	risk	PM

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 13 requests, 17,559 prompt and 1,017 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

```
numbers in the brief that no tool result holds: []
the credit analyst quotes Section 4.12; the quote is in that section word for word: yes; the brief holds the ...
```

The failing check is the credit analyst's own: a figure 2.0 with unit to_1, under the key text.

The PM has no tool. Every number in the brief traces to the specialists' answers; it added nothing.

How parallel tool requests work

The setting	parallel_tool_calls on lets one reply hold several tool requests
The run	The framework runs them together; every result returns in the next request
The effect	Two specialists work side by side
You control	One field of ModelSettings, on a copy of one agent
What comes out	Fewer orchestrator requests, and a log where requests overlap

Parallel tool requests, once

```
parallel_tool_calls: desk False, parallel_desk True
```

2

tool requests in one reply

6

requests, against 7 off

4

replayed from the run before

`the log: orchestrator, risk, macro, risk, macro, orchestrator`

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 6 requests, 7,598 prompt and 368 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Live, the interleaving depends on reply times: two runs can log two orders. So the course keeps it off.

How the cost of a run works

The price	Requests priced by tokens: prompt at one rate, completion at another
The prompt	System message, every tool description, every message so far
The growth	More cost with each request and with each tool held
You control	The requests, the tools held, the model
What comes out	A cost for each run, set side by side

What the orchestrator costs

run	agents	requests	prompt	completion	largest	usd
handoffs	2	4	6610	249	2173	0.003042
agents as tools	3	7	8254	352	2030	0.003865
parallel on	3	6	7598	368	2030	0.003628
one agent	1	3	8066	115	2882	0.003410
desk brief	5	13	17559	1017	2185	0.008651

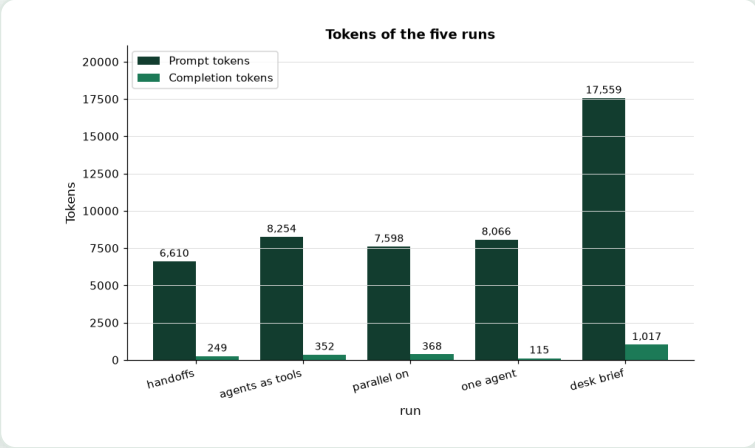
```
numbers in the answer that no tool result holds: ['298,998.53']
sector_totals returned 10 sectors, which add up to 306,996.49 dollars; pandas: 306,996.49
```

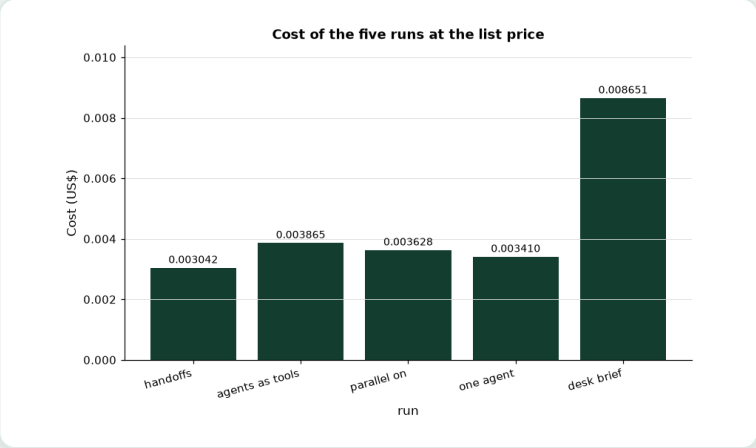
Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 8,066 prompt and 115 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

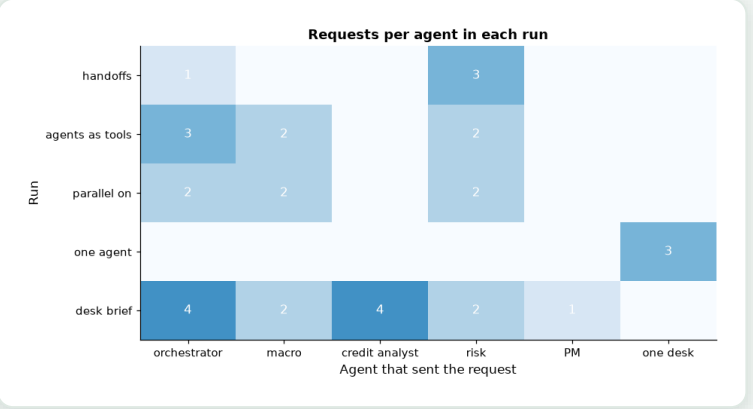
Twenty tools: the one agent took sector_totals and added the sectors itself, short of the key.

The split costs requests, and each request is smaller. Here it bought the right total.

The five runs in pictures





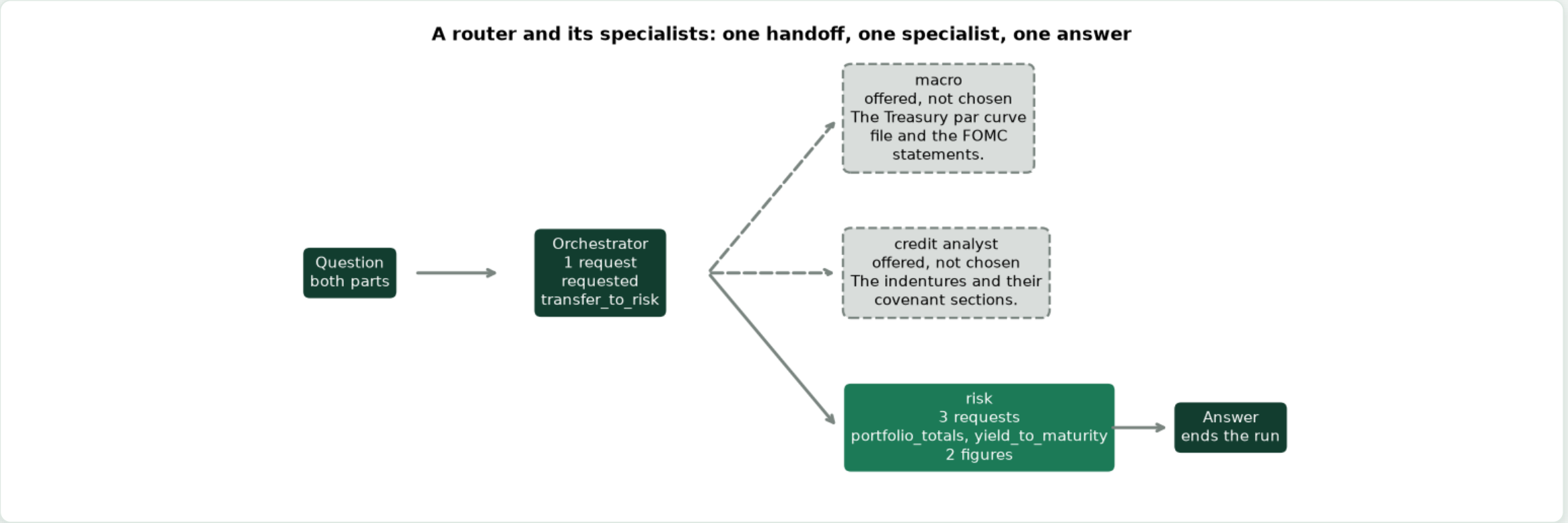


Run	Agents	Requests	Prompt	Completion	Largest	Cost (US$)
handoffs	2	4	6,610	249	2,173	0.003042
agents as tools	3	7	8,254	352	2,030	0.003865
parallel on	3	6	7,598	368	2,030	0.003628
one agent	1	3	8,066	115	2,882	0.003410
desk brief	5	13	17,559	1,017	2,185	0.008651

How a router works

The problem	Which specialist owns this request?
The method	The orchestrator holds one handoff for each specialist and no data tool
The handoff	The conversation passes to the specialist named; its answer ends the run
You control	The descriptions the orchestrator reads, and each specialist's tools
What comes out	One specialist's answer; a two-part request reaches one of them

A router and its specialists, drawn

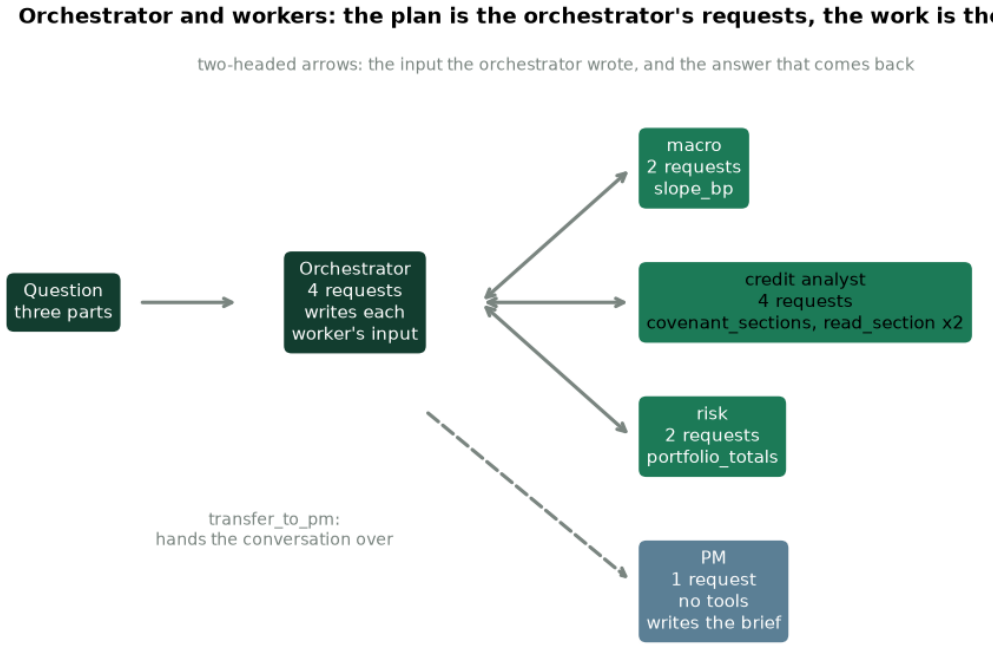


Chosen: risk, 4 requests in the run. The question had two parts; the arrow reaches one box.

How orchestrator and workers works

The problem	Work whose parts cannot be listed before the question is read
The plan	The orchestrator picks the workers and writes each one an input
The workers	Each runs in a conversation of its own and returns an answer
You control	Allow-lists, descriptions, instructions, each worker's step limit
What comes out	The orchestrator's requests are the plan; the workers' are the work

Orchestrator and workers, drawn



Orchestrator 4 requests, workers 8, PM 1. The orchestrator's replies chose the workers, their order, and their inputs.

What the orchestrator wrote for each worker

macro: 2 requests, slope_bp

What is the 2 Yr to 10 Yr slope in basis points on 2026-10-02 in the Treasury par curve file?

credit analyst: 4 requests, covenant_sections, read_section, read_section

What is the Consolidated Fixed Charge Coverage Ratio stated in the indebtedness covenant of the boyd_2021 indenture? Provide the section and the exact words that state it.

risk: 2 requests, portfolio_totals

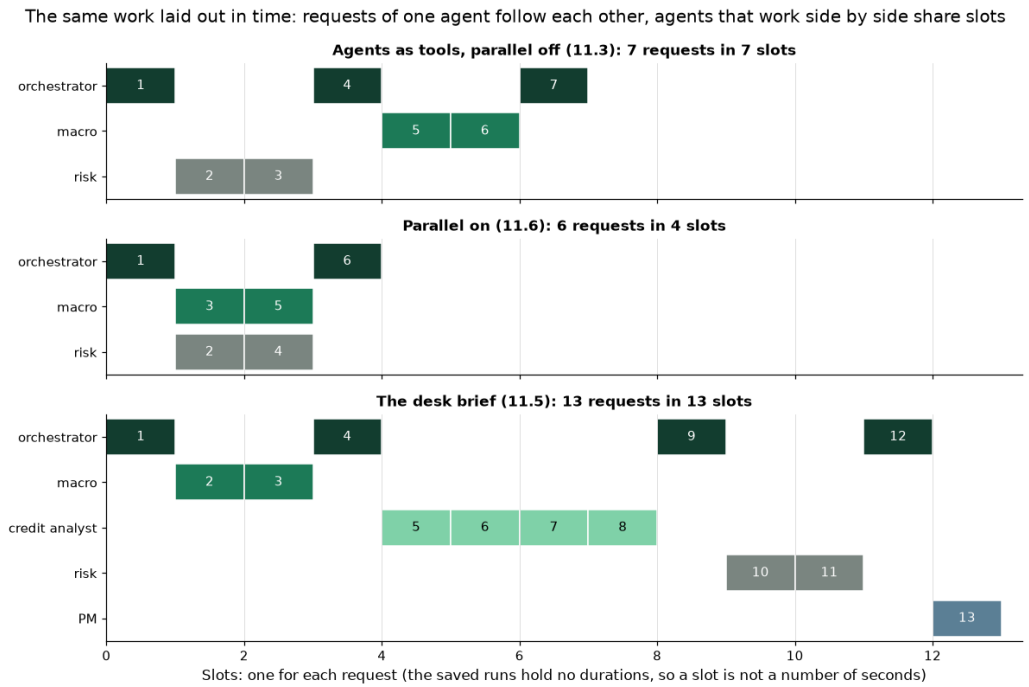
What is the total position DV01 in dollars and the DTS of the synthetic holdings file as of 2026-09-30?

Each input is the orchestrator's rewording of one part of the question. Each worker saw only its input.

How the timeline works

The picture	One bar for each request, in the row of the agent that sent it
Side by side	Overlapping work shares its first slot; a batch lasts as long as its longest member
The unit	A slot is a request: the saved runs hold no durations
You control	Whether the orchestrator may ask for several workers in one reply
What comes out	A count of slots to set against the count of requests

A timeline of the runs



Slots: 7 for the run with parallel off, 4 with it on, 13 for the desk brief. Slots, not seconds.

How the five patterns compare

The question	Who fixes the path: your code, or the model
Code fixes it	Chain, fan-out, evaluator loop: the cost is a count you can state ahead
Model names one	Router: one request to name a specialist, then that specialist's own
Model plans	Orchestrator and workers: the question sets the cost
What comes out	A pattern chosen for the work, with its cost in requests

Five patterns, one table

Chain with a gate	your code: the steps, and a gate between them. Fits when each step needs the one before, and a bad step must stop the rest.	4 (day 9)
Parallelization	your code: the branches, and the aggregator that joins them. Fits when the parts are independent and time matters.	6 (day 9)
Routing	your code lists the specialists; the model names one. Fits when requests fall into kinds, and each kind has its own tools.	13 (day 9), 4 (handoffs, day 11)
Orchestrator and workers	the model: which workers, in what order, with what input. Fits when the parts cannot be listed until the question is read.	7 (two workers), 13 (three workers and the PM)
Evaluator and optimizer	your code: the loop and the rubric; the model drafts. Fits when a good answer can be written as a rubric, and a first draft rarely meets it.	4 (day 9)

Read down the second column: who fixes the path. Requests are those of the course's saved runs.

Excel: requests and tokens by agent with SUMIF

```
=SUMIF($B$2:$B$15,"orchestrator",D2:D15)      sum range E, F, G: tokens and dollars
```

orchestrator	10 requests, 7,588 prompt, 412 completion tokens
risk	9 requests, 17,880 prompt, 632 completion tokens
macro	6 requests, 7,112 prompt, 369 completion tokens
credit analyst	4 requests, 6,277 prompt, 353 completion tokens
PM	1 requests, 1,164 prompt, 220 completion tokens
one desk	3 requests, 8,066 prompt, 115 completion tokens
PM, typed pm	1 requests, 1,164 prompt, 220 completion tokens

```
pm              Excel 1 requests; pandas: no agent named 'pm'
```

Excel 16.0 build 20430, typed by script. SUMIF ignores case ("pm" finds PM); pandas' == does not.

Git: runs.csv by agent, and bring it home

```
agentkit day 11: specialist, route_table, 54 tests; runs.csv by agent
Start day 11: the earlier modules and agentkit as day 10 left it, 52 tests
agentkit.py      | 41 ++++++++++++++++++++++++++++++++++++++++++++
runs.csv         | 15 +++++++++++++++
test_agentkit.py | 24 ++++++++++++++++++++++++
3 files changed, 80 insertions(+)
```

```
python -m pytest -q test_agentkit.py          # 54 passed
git add agentkit.py test_agentkit.py
git commit -m "agentkit day 11: specialist, route_table"
```

One row per agent per run: git log -p runs.csv later shows which agent's numbers changed.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Exactly an allow-list: one job, its own tools, nothing else.

→

2

Code back

Written for the course in the style of an AI answer. It runs.

→

3

Test first

Build all four; assert each one's tools. The PM holds 20, dv01 among them.

→

4

Fix

The allow-list and nothing else, checked: 6, 3, 11, and 0 tools.

Today: specialist() hands every agent all 20 tools "so none gets stuck": the PM can compute a DV01 itself.

Code that runs is not the same as code that is right.

What you can do now

- Why a desk of agents splits the work
- specialist: tools equal to an allow-list, asserted
- Handoffs pass the conversation; tools keep control
- route_table and the request table, side by side
- A brief from an agent with no tools, checked

Week 3, Day 12 Memory and state